

MACE

Multi-core Agentic Co-design Engine

Project Handoff Document

A3 CHIA Hackathon (Google + NVIDIA, MICRO 2026 workshop)

Submission deadline: Sep 20 AoE (~17:00 PKT, Sep 21) — Code freeze: Sep 19

Prepared Sep 8, 2026, for a teammate joining the project in progress

This document was assembled directly from the repository's commit history, its living design-plan document, and the current state of the working tree (including uncommitted work in progress at the time of writing) — not from a summary of a summary. Where something is proven, it says how. Where something is broken or unknown, it says that plainly too. Section 4 (cloud) and the “in-flight” callout at the top of Section 5 cover things that are not yet in the project's own planning document.

Contents

Contents	2
1. Overview	4
The two-phase architecture	4
Where things live	4
2. Architecture, in detail	6
2.1 chia_openpiton/ — the adapter	6
2.2 mace/ — the agentic loop	7
3. What's done, concretely	9
3.1 Test tiers used throughout	9
3.2 Phase 1 — chia_openpiton	9
3.3 Phase 2 — the MACE loop	10
4. GCP / cloud — read this one carefully	12
4.1 The setup	12
4.2 The confirmed bug	12
4.3 What was tried, in order, including a wrong turn that was caught	12
4.4 Where that leaves the question	13
5. What's left	15
5.1 The three baselines the paper needs	16
5.2 The paper itself	16
5.3 GCP: a genuinely open decision, not a foregone conclusion	17

6. How to actually run things	18
6.1 Environment: the WSL / Windows split	18
6.2 First-time setup	18
6.3 Running the test suite	18
6.4 Running a real end-to-end demo	19
6.5 The cluster	19
7. Known rough edges and standing conventions	20

1. Overview

What MACE is. MACE (Multi-core Agentic Co-design Engine) is an agentic loop that autonomously configures, builds, runs, and verifies multicore RTL designs against real hardware simulation, and repairs its own failures. Given a target core, mesh size, an objective in English, and a budget (iterations / wall-clock / dollars), it plans a task DAG with an LLM, dispatches tasks in parallel across real OpenPiton checkouts, verifies every change against real Verilator RTL simulation, diagnoses failures with a second LLM call, feeds that diagnosis back into a replan, and iterates until the design passes or the budget runs out.

Why it exists. This is a submission to the A3 CHIA Hackathon (Google + NVIDIA, MICRO 2026 workshop). Judging criteria are a 4-page paper, an open-sourced [chia](#) loop, and its results. Submission deadline is **Sep 20 AoE** (~17:00 PKT Sep 21); **code freeze is Sep 19**. As of this document, today is Sep 8 — there is real time pressure but not zero runway.

[chia](#) (github.com/ucb-bar/chia) is an existing framework for orchestrating agentic hardware-design workflows on Ray clusters, developed by UC Berkeley's ucb-bar group. It is treated strictly as an external dependency here: checked out separately at [Desktop/chia](#), pinned at a known revision, and **never forked or vendored**. [OpenPiton](#) (github.com/PrincetonUniversity/openpiton) is an open-source many-core RISC-V/SPARC research platform from Princeton, checked out separately at [Desktop/openpiton](#). MACE's job is to make CHIA and OpenPiton talk to each other, then build an agentic loop on top of that connection.

The two-phase architecture

Phase 1 — [chia_openpiton/](#): a CHIA *platform adapter* for OpenPiton. It exposes `configure / build (Verilator) / run` (simulate a test program against the built RTL) / `collect` through one interface, deliberately shaped like CHIA's other platform adapters (e.g. [chia/esp](#)) so that it is realistically upstreamable as a PR to the CHIA project itself. [chia_openpiton](#) imports **nothing** from [mace](#) — that one-way dependency boundary is a hard rule the whole project follows, checked by an actual test (`assert 'mace' not in sys.modules`).

Phase 2 — [mace/](#): the agentic loop layered on top of the Phase-1 adapter. This is the project's actual novel contribution — the pipeline the paper argues for: *Requirement* → *Planning* → *Task Decomposition* → *Parallel Agent Execution* → *RTL/System Integration* → *Verification* → *Failure Analysis* → *Iteration*.

Composition rule: [examples/](#) is the **only** place in the repo allowed to depend on both halves — e.g. [examples/mace_end_to_end.py](#), a real runnable script wiring `plan` → `execute` → `metrics` together. Neither [chia_openpiton/](#) nor [mace/](#) is allowed to reach across into a driver script's concerns.

Where things live

Path	What it is
C:\Users\Potato\Desktop\MACE	This repository. Git repo, branch main , currently 8 commits ahead of origin/main (not yet pushed).
C:\Users\Potato\Desktop\openpiton	OpenPiton checkout (separate git repo), mounted in WSL as <code>/mnt/c/Users/Potato/Desktop/openpiton</code> . HEAD 1c6bfd2 , Ariane submodule pinned 4c01614 .
C:\Users\Potato\Desktop\chia	CHIA checkout (separate git repo), the framework dependency. Installed editable (<code>pip install -e</code>).
MACE\chia_openpiton\	Phase 1: the CHIA/OpenPiton adapter.

Path	What it is
MACE\mace\	Phase 2: the agentic loop.
MACE\examples\	Runnable driver scripts; the only place depending on both halves.
MACE\cluster\	CHIA cluster YAML configs (WSL head + GCP workers over Tailscale).
MACE\scripts\	Standalone diagnostics and the checkout patch script.
MACE\paper\	The 4-page paper. <i>Does not exist yet as of this writing — see Section 5.</i>

Every path above was confirmed directly against the filesystem/git while preparing this document, not copied from an older plan.

2. Architecture, in detail

This section is grounded in the actual current source, not a description of an earlier design. Where the implementation diverged from the original plan, that is called out explicitly — several pieces described in early planning (e.g. `chia_wait/TrackedRef` for fan-out, a pinned Verilator version) were tried and deliberately replaced with something simpler once real hardware testing showed the simpler thing was enough.

2.1 chia_openpiton/ — the adapter

Module	Role
<code>state_def.py</code>	Result/config dataclasses. <code>PitonConfig</code> (frozen: core, mesh, network topology, cache geometry, RTL defines, extra flags) carries a <code>.key</code> — a sha256 of everything that changes the produced model (source revisions, Verilator version, config, and any file-level diff) — and a derived <code>.build_id</code> . <code>PitonBuildArtifact</code> , <code>PitonRunResult</code> (success is derived from the simulator's transcript verdict, <i>never</i> from the process exit code — an RTL sim exits 0 whether the program passed or not), <code>PitonRegressResult</code> , <code>PitonCollectResult</code> .
<code>parse.py</code>	Pure regex/text parsers, zero I/O, zero subprocess — testable against captured log fixtures with no toolchain present. <code>sim_verdict()</code> reads <code>sim.log</code> for the <code>PASS (HIT GOOD TRAP) / FAIL(...)</code> / maxcycles markers; <code>status_diag/cycles/exec_cycles()</code> read <code>status.log</code> ; <code>build_failure_reason()</code> tags a build failure by cause; <code>needs_no_timing()</code> decides a Verilator-version-dependent build flag; <code>diaglist_group()</code> parses OpenPiton's own semi-XML regression-group format.
<code>openpiton_workspace.py</code>	<code>OpenPitonWorkspaceNode(CollocatedNode)</code> — one checkout, placement-group-pinned so <code>configure→build→run</code> always land on the same worker (state lives on disk under <code>\$PITON_ROOT/build/manycore/<build_id>/</code>). Members (<code>configure/build/run/regress/sims/put_file/collect/clean</code>) are <code>@staticmethods</code> taking <code>piton_root</code> first; a <code>_RootBoundChiaFn</code> wrapper rebinds instance calls so <code>node.build.chia_remote(cfg)</code> auto-supplies the root. <code>build()</code> skips re-invoking <code>sims</code> entirely when an identical config already built successfully (a disk marker, not CHIA cache machinery) — measured 104s → 0.00s on a repeat build.
<code>tools.py</code>	<code>PitonToolServer(AsyncJobTool)</code> — the agent-facing MCP surface. Wraps the node's members <i>locally</i> (no extra Ray hop). Build/run go through <code>AsyncJobTool</code> (start + poll) because they are minutes long and would otherwise strand a blocking MCP call; <code>grep/collect/config_get/config_set</code> are fast synchronous tools.
<code>test/</code>	143ish tier-0 unit tests (see §3 for the exact current count) plus <code>test/cluster/</code> tier-1/2 tests gated by env vars, and <code>test/fixtures/</code> — real captured logs (pass, bad-trap fail, timeout, maxcycles, build errors) used so the parsers are tested against ground truth, not invented text.

The call chain for one adapter operation

```

node = OpenPitonWorkspaceNode("/path/to/openpiton") # binds piton_root, reserves a placement gro
cfg = get(node.configure.chia_remote(x_tiles=1, y_tiles=1, core="ariane"))
art = get(node.build.chia_remote(cfg)) # sims ... -vlt_build -build_id=<cfg.build_id>
res = get(node.run.chia_remote(cfg, "hello_world.c"))
assert res.verdict == "pass" # read from sim.log, NOT from res.returncode

```

2.2 mace/ — the agentic loop

Module	Role
<code>spec.py</code>	<code>MaceSpec</code> (frozen: workloads, objective, core, target_mesh, budget), <code>Budget</code> (frozen: max_iterations / max_usd / max_wall_s — three <i>independent</i> caps, all enforced), <code>Task</code> (frozen: id, deps, kind, spec), <code>StepResult</code> (one task's outcome: query + build + run + passed), <code>Triage</code> (diagnosis + fix), <code>LoopResult</code> (a whole run's outcome).
<code>agents.py</code>	Footer-line parsers. The Planner and triage agent talk back to the loop through plain <code>TASK: / DIAGNOSIS: / FIX:</code> lines embedded in ordinary prose — no CHIA backend guarantees a JSON mode. Deliberately permissive (regex over the whole response, case-insensitive, malformed lines dropped not raised) because a real LLM response is prose with directives in it, not a form.
<code>loop.py</code>	<code>run_mace_step()</code> — the smallest real slice: one task, no fan-out, no Planner. One LLM turn → build → run the first gate workload → pass/fail. This proves the wiring (LLM dispatch → build → run → gate) independent of any planning intelligence.
<code>integrator.py</code>	<code>topological_levels()/topological_order()</code> (dependency-level sort with cycle detection). <code>integrate()</code> — one shared checkout, fully serial, stops at the first failed task. <code>integrate_parallel()</code> — one <code>OpenPitonWorkspaceNode</code> per checkout, dispatches a whole level's tasks (prompt, then build, then run) before resolving any of them, batched to <code>len(piton_roots)</code> at a time; stops at the first failing <i>level</i> .
<code>planner.py</code>	<code>plan(spec, llm, feedback)</code> — one LLM call → <code>parse_tasks()</code> → validated as a real DAG via <code>topological_levels()</code> . <code>feedback</code> (from a previous iteration's triage) is appended as extra prompt context for a replan; empty on a first attempt. Raises <code>PlanningError</code> on no usable output.
<code>trriage.py</code>	<code>trriage(step_result, llm)</code> — one LLM call given the failing build/run's own log tail, asking for a <code>DIAGNOSIS: / FIX:</code> footer. Read-only tools only — grep/collect over logs, never a write tool. Any actual fix is a future <i>Planner</i> task, not something triage does itself.
<code>orchestrator.py</code>	<code>run_mace_loop()</code> — the top-level entry point; see the call-chain walkthrough below.
<code>metrics.py</code>	<code>SQLiteNode</code> -backed record: <code>runs/iterations/tasks/failures</code> tables, every write <code>INSERT OR REPLACE</code> keyed by full primary key (idempotent under a retried/resumed run). <code>summary()</code> derives the five metrics the proposal promises (successful tasks, iterations, failures recovered, execution time, compute \$) from these tables at read time.
<code>llm.py</code>	<code>make_llm()</code> picks a backend from the <code>MACE_LLM</code> env var (<code>opencode</code> default, or <code>claude/antigravity/vertex</code>), each lazily imported. <code>extract_cost_usd()</code> reads a per-call dollar cost where the backend's <code>QueryResult</code> carries it (OpenCode, Antigravity survive a remote round-trip); always returns 0.0 for Claude — a documented, real API-shape limitation (Claude's cost lives on the LLM instance's own state, invisible after remote dispatch), not silently wrong.

Module	Role
<code>replay.py</code>	Wraps CHIA's cache/bypass mechanism, scoped honestly: it can only ever replay a tagged call's <i>return value</i> , never a side effect — so an LLM's file edits (made via a Bash tool) can never be replayed, only its query text, and only a build/run's pass/fail artifact. Confirmed by reading <code>chia/base/cache.py/bypass.py</code> directly. <code>tag_for()</code> builds a stable cache key; only <code>integrate_parallel</code> 's remote-dispatch call sites can be tagged at all (a local call, which is all <code>run_mace_step</code> and <code>integrate</code> ever make, never touches this machinery).
<code>workloads.py</code> + <code>workloads/</code>	Three real gate C programs (<code>barrier_atomic.c</code> , <code>producer_consumer.c</code> , <code>scatter_gather.c</code>) plus a <code>CHECKSUMS</code> file. <code>verify_checksums()</code> is the loop's self-tamper check: a task that edited its own pass/fail oracle (by accident or otherwise) must not get to grade its own work.

A real, non-obvious hardware bug shaped `workloads.py`: on this RTL/toolchain combination, a plain *volatile* read of a shared *array element* right after a write to it is **not reliably visible** — even from the same hart, a few instructions later — while the identical value read back through an atomic fetch-op always is. Found by narrowing a “computes the right value, then hangs forever” symptom down to a 10-line repro. Fix: every workload routes every shared read through an `atomic_read()` helper, never a plain load. Documented in the module docstring so nobody rediscovers this by hand. Multi-tilde coherence itself — the property these workloads exist to stress — remains unverified on real multi-tilde hardware; see Sections 4 and 5.

The call chain for one full loop run

This is the actual sequence `examples/mace_end_to_end.py` triggers, module by module:

```

main()                                     # examples/mace_end_to_end.py
ray.init(resources={"openpiton": N, "opencode_creds": N})
llm = mace.llm.make_llm("opencode", model=...)
db = mace.metrics.open_db(db_path)
result = mace.orchestrator.run_mace_loop(piton_roots, spec, llm, db)
run_id = mace.metrics.start_run(db, spec)
mace.workloads.verify_checksums()           # stop immediately if the gate itself was e
for iteration in range(spec.budget.max_iterations):
    # -- budget check (wall_s, then usd) happens here, BEFORE the iteration starts --
    tasks = mace.planner.plan(spec, llm, feedback) # 1 LLM call -> TASK: lines -> validated D
    results = mace.integrator.integrate_parallel(   # tagged with run_id/iteration for replay
        piton_roots, spec, tasks, llm, run_id=run_id, iteration=iteration)
    # per dependency level, per batch of len(piton_roots) tasks:
    #   llm.prompt.chia_remote(...) -> build.chia_remote(...) -> run.chia_remote(...)
    #   (every call in a batch is DISPATCHED before any of them is RESOLVED)
    mace.metrics.record_iteration(db, run_id, iteration, results, wall_s, usd)
    if all(r.passed for r in results):
        status = "passed"; break
    failed = first non-passing result
    diagnosis = mace.triage.triage(failed, llm)    # 1 LLM call -> DIAGNOSIS:/FIX:
    mace.metrics.record_failure(db, run_id, iteration, failed.task.id, diagnosis)
    feedback = f"Task {failed.task.id} failed: {diagnosis}" # -> next iteration's plan() cal
    if status == "passed" and any earlier iteration recorded a failure:
        mace.metrics.mark_all_recovered(db, run_id)
    mace.metrics.finish_run(db, run_id, status)
    return LoopResult(run_id, status, iterations)
print(mace.metrics.summary(db, result.run_id))    # the five promised metrics

```

Every line above is a real call in the current source (`mace/orchestrator.py`, `mace/integrator.py`) — this is not a simplified paraphrase.

3. What's done, concretely

“Proven” is used carefully in this section — the project draws a real distinction between code that is only exercised by a stub/fake backend (tier 0/1) and code that has actually run against genuine Verilator RTL simulation and a genuine LLM (tier 2). Both matter, but they answer different questions: tier 0/1 proves the *wiring* is correct; tier 2 proves it *actually works*.

3.1 Test tiers used throughout

Tier	Needs	Proves
0 — local, no Ray	Nothing real; a stub <code>sims</code> script and a stub checkout factory	Parsers against real captured log fixtures; config validation; exact CLI argv shape per operation; error paths (timeouts, path escapes, junk-directory regressions)
1 — live Ray, stub tool	A local Ray cluster with fake custom resources; the stub <code>sims</code> staged on a worker	Real placement-group pinning, real remote dispatch/fan-out, real cache/bypass round-trips — but a <i>fake</i> hardware verdict
2 — real tool	<code>OPENPITON_TEST_REAL=1</code> + a real OpenPiton checkout + Verilator (+ Ariane toolchain)	Genuine RTL builds and genuine simulated pass/fail verdicts on real hardware
loop (FakeLLM)	Nothing real — a scripted response queue standing in for any LLM backend	Loop control flow (plan → dispatch → gate → triage → replan) independent of any real model's behavior

3.2 Phase 1 — chia_openpiton

Phase 1 is complete. `state_def.py`, `parse.py`, `openpiton_workspace.py`, `tools.py` are all real and tested. Running the tier-0 suite directly while preparing this document (`pytest chia_openpiton/test mace/test -q`, excluding the `cluster/` tier-1/2 subdirectories) gave **309 passed, 0 failed**, in about 9 seconds — the project's own planning doc cites “143 tests” for `chia_openpiton` as of Sep 7; the current tree (after Sep 8's changes) has ~137 `def test_` functions there and ~162 more under `mace/test`, all currently green.

#	Acceptance test	Status
1	Local Ariane: <code>configure(1,1)</code> → build → <code>run(hello_world.c)</code> → verdict == pass	REAL HARDWARE — proven
2	GCP: 2×2 Ariane build on a real GCP worker	BLOCKED — see Section 4
3	Fan-out: 3 tests across 2 checkouts in parallel	REAL HARDWARE — proven, 176s vs 324s serial
4	<code>chia viz</code> renders the example script's task graph	proven (required splitting dispatch/get into separate statements)
5	<code>pytest chia_openpiton/test</code> green on fixtures	proven, currently green

4 of 5 acceptance tests are real-hardware-proven; #2 is the one open item, and it is a cloud/infrastructure problem, not an OpenPiton or adapter problem — see Section 4.

Two real, non-obvious bugs were found only by running the real toolchain, and are now permanently patched (idempotent, in `scripts/patch_openpiton.sh`): binutils 2.38+ split `zicsr/zifencei` out of base RV64I, breaking the boot ROM's own 2019-era assembly; and GCC 15+ defaults to C23, under which `void init_uart();` means zero arguments rather than “unspecified,” breaking the boot ROM's actual 2-argument call to it — this one hid behind a stale `.o` file for an entire build cycle before surfacing.

Docker was built, then dropped entirely. Docker Desktop proved unstable on this machine (a real WSL integration crash, not a config mistake) and turned out to be unnecessary — CHIA's `available_node_types` are container-optional, and the bare-VM `setup_commands` pattern (used for the GCP worker too) is simpler and is the validated path. `dockerfiles/openpiton-chia.Dockerfile` still exists but nothing depends on it.

No Verilator version is pinned. The original plan called for pinning 4.014; what shipped instead detects the *worker's actual* Verilator major version at build time and adds `--no-timing` only for v5 — proven correct on both a v5.049 devel build (this WSL host) and v4.038 (the now-parked Docker image) building the identical Ariane design.

3.3 Phase 2 — the MACE loop

All 11 steps of the planned build sequence landed, each as its own small, individually tested, individually committed change, TDD throughout. The loop has been run for real **twice** end to end via `examples/mace_end_to_end.py` against actual Ariane hardware simulation: once with `opencode/big-pickle` (which timed out three times and correctly fell back to a hand-written task that still passed — proving the deterministic build/run/gate path does not depend on LLM availability), and once with `opencode/nemotron-3-ultra-free` (which produced a genuine multi-task dependency chain, every task passing on real hardware, fully recorded via `mace.metrics`).

Precision on the fail → diagnose → replan → pass cycle

The *mechanism* for detecting a failed gate, recording a diagnosis, feeding it back into the next `plan()` call, and marking earlier failures recovered once a run eventually passes is real, wired end-to-end, and **proven** — but that specific proof (`mace/test/cluster/orchestrator_e2e_test.py`, commit `38e6a7a`) uses a **stateful STUB** `sims` (fails its first run, passes every run after), i.e. real Ray dispatch and real control flow, but a *fake* hardware verdict — tier 1, not tier 2.

The two *real-Ariane-hardware* runs of `mace_end_to_end.py` described above are genuine tier-2 proof of the happy-path DAG (plus one real LLM-timeout recovered via fallback) — but neither of those two real-hardware runs is documented as having hit an actual RTL build/run gate *failure* that then got triaged and replanned to a subsequent real-hardware pass. That exact combination (real hardware **and** a real gate failure recovered by triage) has not yet happened, or at least is not yet evidenced anywhere in the commit history or planning doc. Worth doing deliberately for the paper's own credibility — it is very likely achievable (the mechanism is proven, and gate workloads were deliberately designed to fail on naive configs) but it has not happened yet.

A real gap was caught by real hardware, not by its own tests: `run_mace_step` originally never passed an explicit `rtl_timeout` to `run()` — invisible to its tier-0 tests (the stub `sims` ignores timing entirely), so it stayed hidden until real hardware fan-out testing made every task time out. Fixed by importing `RECOMMENDED_RTL_TIMEOUT`; a new tier-0 argv-shape regression test now catches this class of bug without needing real hardware again.

Planned infrastructure turned out unnecessary: `chia_wait/TrackedRef` (named explicitly in the original plan for fan-out) solve a different problem (a wedged raylet) than fan-out itself needs. `integrate_parallel` uses the same plain dispatch-then-collect shape already proven by `chia_openpiton`'s own fan-out acceptance test — proven again here: 30s parallel vs. 57s serial-equivalent for two real tasks across two real checkouts.

The replay question is answered, not just deferred. Direct reading of `chia/base/cache.py/bypass.py` confirmed structurally: CHIA's cache/bypass mechanism captures a `.chia_remote(...)` call's *return value only* — there is no side-effect capture anywhere in CHIA, and nothing upstream wires this to an LLM or build node today. `mace/replay.py` is scoped honestly around that limit and is proven end-to-end on real Ray cache/bypass actors. It cannot, and structurally never will, replay the actual RTL edits an agent made through a tool — git remains the record of what actually changed.

The three gaps flagged in the previous planning pass are closed (Sep 8, each its own tested, committed change): *cost tracking* (`Budget.max_usd` is now actually enforced, checked before each iteration the same way `max_wall_s` already was; the dollar tally is a documented lower bound — per-task execution cost only, not the Planner's or triage's own call cost); *checksum verification* (`verify_checksums()` runs before any LLM call or checkout is touched; a mismatch stops the run immediately with `status="checksum_mismatch"`); and *replay tagging* (`run_mace_loop` now threads its own `run_id/iteration` through `integrate_parallel` so a real run is tagged for replay automatically, with zero extra caller effort).

4. GCP / cloud — read this one carefully

This is the part of the project that needs the most explaining, so it gets its own section and a blow-by-blow account. The short version: **credits landed, a real multi-machine cluster comes up reliably, but a task the cluster's scheduler assigns to the GCP worker never actually reaches it** — and this looks like a bug in CHIA's own cloud-relay code, not in this project's code. **This does not block the paper's core contribution**, which is proven entirely without any cloud dependency (Section 3). Whether to keep chasing it is a genuinely open decision — see Section 5.

4.1 The setup

Credits landed on GCP project `mace-508004`. The cluster (`cluster/local.yaml`) has two node types: `openpiton_local` (this WSL machine, 2 `openpiton` slots, the environment already validated for everything in Section 3) and `openpiton_gcp` (one `e2-standard-8` GCP worker, 8 `openpiton` slots — not the originally-planned `c2-standard-30`, because this fresh GCP project has a 12-vCPU total default quota and zero quota for the C2 family specifically, both confirmed by direct `gcloud` attempts, not assumed). Because the WSL head sits behind NAT with no public IP, the two machines join a `Tailscale` tailnet and CHIA relays Ray traffic over it rather than addressing nodes directly. Both node types are bare VMs — no Docker (Section 3's Docker-drop decision applies here too).

This has come up successfully, multiple times: head + local worker + GCP worker all tailnet-join and report into Ray with the expected resource shapes (2 vs. 8 `openpiton` units), repeatedly, across several separate `chia up` sessions. Provisioning the GCP VM itself has needed persistence on at least one occasion — see 4.3.

4.2 The confirmed bug

A task that Ray's scheduler assigns to the GCP worker **never actually executes there**. This is narrower than “the cluster doesn't work”: the GCP worker's raylet registers with the head just fine and shows `Alive` with correct resources the entire time — that direction (worker → GCS) works. The problem is specifically the *other* direction: a task lease the GCS assigns to that worker never seems to reach its raylet, so no worker process is ever even *asked* to start.

4.3 What was tried, in order, including a wrong turn that was caught

- **A real, permanent adapter bug found and fixed.** `OpenPitonWorkspaceNode.__init__` validated `piton_root` as a real *local* directory unconditionally — which can never be true on a genuine multi-machine cluster, where the checkout only exists on the worker that will run against it. Fixed with an explicit `root_on_remote_worker=True` constructor kwarg (deliberately *not* inferred from `require_colocated`, which the test suite already used for an unrelated reason — conflating the two broke an existing regression test on the first attempt, caught by re-running the suite before committing). This fix is real, committed, and unrelated to the scheduling bug below.
- **Placement had to be forced by hand.** Both node types share the resource name `openpiton` (2 local slots, 8 GCP slots), which Ray treats as fungible — CHIA's own placement-group pinning only reserves resource *shape*, never a specific machine. Worked around with `NodeAffinitySchedulingStrategy`, matched by resource *count* rather than IP (the tailnet relay rennumbers node-manager addresses to local loopback-style addresses, making IP matching fragile).
- **First diagnosis attempt — later shown to be wrong.** A forced-placement task never got a Python worker started on the GCP node; a raw `socket.create_connection` to port 6379 on the VM got

`ConnectionRefusedError`; `ss -tlnp` showed only a fixed worker-port range listening, not 6379. Read *at the time* as “the tailnet relay isn't forwarding the GCS port for new connections.”

- **The correction — caught before it misled anything further.** Re-reading CHIA's own `chia/cluster/tailnet.py` afterward (done for free, off the clock, cluster already torn down) showed the real design: Ray's gRPC on every tailnet node is routed through a **single HTTP CONNECT proxy** (`RAY_grpc_enable_http_proxy=1/grpc_proxy` env vars set in each node's own startup script), not one dedicated listener per service. Under that design, the absence of a port-6379 listener in `ss -tlnp` proves *nothing* either way, and the raw socket test exercised a code path Ray's own gRPC client was never going to take. The one fact that still stood on its own: **zero Python worker processes ever started** on the GCP node. Why was still genuinely open — including the possibility the diagnostic was simply killed too early (~90s) before Ray's own retry logic could resolve things.
- **The patient retest — the finding that actually held up.** Re-ran the diagnosis with real patience: `scripts/patient_gcp_dispatch_test.py` (kept, committed, reusable — not a throwaway) forces placement onto the GCP node via a resource count only that pool can satisfy, and polls `ray.wait()` every 10 seconds for up to **6 minutes** instead of blocking or guessing, so a slow-but-working result would stay visible instead of tempting an early kill.
- **A second false lead, caught the same way.** An SSH-side watcher found a process via `pgrep -f default_worker.py` within 10 seconds, environment variables looking correct — seemingly confirming the proxy plumbing worked. It didn't: the *raylet's own* command line contains the literal string `default_worker.py` (it's baked into its `--python_worker_command=` template argv), so the match was the raylet itself, not a spawned child. The correct check — `ps --ppid <raylet_pid>` — showed only `DashboardAgent/RuntimeEnvAgent` as children, **no worker at all**, for the entire 6-minute window.
- **The confirmed finding.** A forced-placement task sat pending for the *full* 360-second budget — a real, waited-out hang, not a premature kill. The GCP raylet's own state-dump, taken *during* that same window, showed `num PYTHON pending start requests: 0`, `num PYTHON pending registration requests: 0`, and every `process_failed_*` counter at 0 — meaning the raylet's WorkerPool was **never even asked** to start a worker for this task. Nothing failed partway; nothing was attempted at all.

4.4 Where that leaves the question

The gap is now precise: it sits somewhere between “*the GCS decides this task's lease goes to the GCP node*” and “*that decision ever reaches the GCP node's own raylet.*” Whatever channel carries a lease *assignment* from the GCS to a specific raylet behaves differently under this tailnet-relay topology than the channel the raylet uses for its own outbound registration/heartbeats — which is the one direction confirmed to work throughout. This reads as a genuine bug (or at least an unsupported configuration) in CHIA's own tailnet-relay / cloud-scheduling plumbing, not something fixable from this repo's own code — and per this project's own rule, CHIA is a dependency that is never forked or vendored, so a real fix belongs upstream.

Not yet done: packet-level tracing on both ends (the head's relay process and the GCP node's own raylet) to see whether a lease-assignment RPC is sent and lost, or never sent at all. This is explicitly called out as a bigger, more deliberate undertaking than fits inside a short paid-VM diagnostic window — the natural next concrete step for whoever picks this up, not something attempted opportunistically.

The cluster was torn down cleanly both times this was investigated (`chia down`, then `gcloud compute instances list` confirmed zero running instances) — no GCP compute time was spent on an actual 2x2 build attempt itself, only on this diagnosis.

What's committed and ready to go the moment this is answered

`chia_openpiton/test/cluster/openpiton_e2e_test.py::TestAcceptance2` (gated behind `OPENPITON_TEST_GCP=1`), the `root_on_remote_worker` fix, and `scripts/patient_gcp_dispatch_test.py` are all correct, committed, and reusable as-is — no further changes are needed on this repo's side either way, only an answer to the scheduling question above (or a decision to stop chasing it — see Section 5).

5. What's left

Right now, uncommitted: the first-ever 2×2 (multi-tile) Ariane build attempt

This is **not** in the project's planning document — it happened after that document's last edit, and the working tree still has it uncommitted as of this writing. Included here so it isn't lost between sessions.

What it is: `scripts/local_2x2_build_test.py` (untracked, new) is a local stand-in for GCP acceptance test #2's technical goal (multi-tile Ariane coherence), run directly against the local WSL checkout: `configure(x_tiles=2, y_tiles=2, core="ariane", extra_flags=zicsr/zifencei) → build → run("hello_world_many.c", finish_mask="1111", rtl_timeout=10_000_000)`. Its own docstring says plainly: “Every real run so far (this whole project) has only ever been 1x1. This is the first attempt at anything larger.”

A real bug was found and fixed (also uncommitted): `scripts/patch_openpiton.sh` gained a third fix. Windows-mounted checkouts (WSL over `/mnt/c`) default `core.symlinks=false`, so git-tracked symlinks (e.g. the baremetal bootrom's device-tree source) materialize as plain text files containing their own target path, instead of real symlinks. This is invisible on a 1×1 build — nothing opens those paths — and only surfaced when a multi-tile build first exercised the baremetal bootrom and `dtc` choked trying to parse a path string as a device tree. The fix detects broken symlinks and re-checks them out for real.

This moved twice more while this document was being written — timeline, precisely: the symlink fix got the baremetal bootrom's device tree compiling cleanly, but the build then hit a second, different failure compiling the *Linux-variant* bootrom (`bootrom/linux/Makefile:37`): `riscv64-unknown-elf-gcc: error: unrecognized command-line option '-mexplicit-relocs'`. A real diagnostic (`scripts/env_diag.py`, dispatched as an actual Ray task so it reproduces the exact failing worker environment rather than a guess in an interactive shell) was built to chase it. That led to a **fourth fix** in `scripts/patch_openpiton.sh`: the same Windows-mounted-checkout root cause as the symlink issue, different symptom — script files with a CRLF-terminated shebang line (`#!/usr/bin/env python3\r`) fail with `No such file or directory` since `env` looks up a program literally named `python3\r`. Swept across 66 affected files, mostly inside the Ariane submodule (one of them a PLIC register-map generator script the multi-tile build actually needs). A cleanup script (`scripts/_fix_split_cflags.py`) also appears alongside it, repairing a Makefile left in a malformed state by an earlier patch attempt.

```
MAX_HARTS = 4
riscv64-unknown-elf-gcc: error: unrecognized command-line option '-mexplicit-relocs';
                                did you mean '-mexplicit-relocs'?
make: *** [Makefile:37: src/main.o] Error 1
sims: Caught a SIG Error compiling rv64 DTS.
```

As of literally the final check before this PDF was generated: the build succeeded, and a run is live

After the CRLF fix landed, the very *next* build attempt against this exact config succeeded for real — confirmed three independent ways on disk: a `.mace_build_ok` marker (only ever written after `OpenPitonWorkspaceNode.build()` itself confirms success), a populated `obj_dir/` with real compiled Verilator objects, and a real `obj_dir/Vcmp_top` binary. Whether the CRLF fix directly caused the `-mexplicit-relocs` error to stop happening, or something else changed between attempts (a stale intermediate object file, the same class of problem fix 2 already hit once — see `patch_openpiton.sh`'s own comments), is **not confirmed either way** — the Makefile line defining that flag is byte-for-byte unchanged, so treat the connection as a strong correlation in time, not a proven mechanism.

A run of `hello_world_many.c` against the real 2×2 mesh was then launched, and at the moment of the last check made while preparing this document, its Verilator process (**PID 159077**) was still actively **running** — about 8 minutes of wall time and climbing, with its instruction trace file still growing (19.6 MB and increasing), which rules out a silent hang. What it has shown so far is genuinely encouraging for a first-ever attempt: reset completed cleanly, and `fake_uart.log` shows “Hello world, this is hart 0 of 4 harts!” — real evidence of a multi-hart boot on this design for the first time in the project. `sim.log` has **no PASS/FAIL verdict line yet** as of that check, so the actual outcome (pass, fail, or a timeout against the 10,000,000-cycle budget) is **genuinely unknown as this document goes to print**.

What to do next: check `build/manycore/mace_739417d3ad77/runs/hello_world_many.c-63910/sim.log` for a `Simulation -> PASS/FAIL` line before doing anything else with this build — it may well have resolved itself by the time you read this. If it timed out or failed, the captured `sim.log/fake_uart.log/trace_hart_00.dasm` are sitting there ready for exactly the kind of log-driven triage `mace/triage.py` automates. Either way, none of this — the symlink fix, the CRLF fix, or the run's actual outcome — is committed yet; that is real, valuable work sitting in the working tree that should not be lost.

5.1 The three baselines the paper needs

The proposal commits the paper to a three-way comparison (planning doc §6.3). As of this writing, **none of the three exist yet as dedicated scripts** — only the full MACE driver (`examples/mace_end_to_end.py`) and two smaller demos exist in `examples/`. All three need to share the same gate workloads, the same metrics schema, and the same failure taxonomy for the comparison to mean anything.

- (a) **Manual mesh scaling by hand** — $1\times 1 \rightarrow 2\times 2 \rightarrow 4\times 4$, with a time log, no agent involved.
- (b) **One-shot LLM configuration, no tools** — a single LLM call proposes a configuration with no ability to inspect logs or iterate, contrasted against MACE's tool-driven multi-turn loop.
- (c) **The full MACE loop** — already built; just needs to be run under the same conditions as (a) and (b) for a fair comparison.

5.2 The paper itself

`paper/` does **not exist yet** as a directory in the repository as of this writing. The 4-page paper is entirely ahead of the team, due before the Sep 19 code freeze along with everything else. `CHIA_proposal.pdf` (the original proposal) is the closest existing document to draw from.

5.3 GCP: a genuinely open decision, not a foregone conclusion

Three real options exist for acceptance test #2, and none is obviously correct under the remaining time budget — this is presented deliberately as an open decision for the team, not a recommendation baked in here:

- **Keep chasing it.** Do the packet-level trace (Section 4.4) to pin down whether CHIA's lease-assignment RPC is sent-and-lost or never-sent, and fix or patch around it locally. Risk: explicitly called out as a bigger undertaking than a short diagnostic window, competing directly with paper-writing time in the final runway.
- **Report it upstream.** Hand the precise, evidence-backed question (Section 4.4's last paragraph) to CHIA's maintainers or file it as an issue. Lower time cost, but resolution is out of this team's hands and almost certainly won't land before Sep 19.
- **Accept it as a documented limitation.** Ship the paper on the strength of Section 3's real, cloud-independent results, and describe the cloud gap honestly (as this document does) rather than silently. The core contribution does not need this to be resolved.

6. How to actually run things

6.1 Environment: the WSL / Windows split

Checkouts live on the Windows filesystem but the actual toolchain (Verilator, the RISC-V GCC, `sims`, the conda environment) runs inside **WSL Ubuntu** — Windows paths are reached from WSL as `/mnt/c/...`. This split is exactly why the symlink bug in Section 5's in-flight callout exists: a Windows-mounted git checkout defaults `core.symlinks=false`.

Checkout	Windows path	WSL path
MACE (this repo)	<code>C:\Users\Potato\Desktop\MACE</code>	<code>/mnt/c/Users/Potato/Desktop/MACE</code>
OpenPiton	<code>C:\Users\Potato\Desktop\openpiton</code>	<code>/mnt/c/Users/Potato/Desktop/openpiton</code>
CHIA (dependency)	<code>C:\Users\Potato\Desktop\chia</code>	<code>/mnt/c/Users/Potato/Desktop/chia</code>

The conda environment is named `chia_env` (conda-forge, Python 3.10.19) and lives inside WSL. All commands below assume you are in a WSL shell with it active, in the MACE repo root:

```
wsl -d Ubuntu
source ~/miniconda3/etc/profile.d/conda.sh && conda activate chia_env
cd /mnt/c/Users/Potato/Desktop/MACE
```

6.2 First-time setup

```
conda create -n chia_env -c conda-forge --override-channels python=3.10.19
conda activate chia_env

git clone https://github.com/ucb-bar/chia.git ~/.Desktop/chia # or wherever you keep it
pip install -e /mnt/c/Users/Potato/Desktop/chia # not on PyPI at the pinned revision

cd /mnt/c/Users/Potato/Desktop/MACE
pip install -e ".[test]"

# Apply once per FRESH OpenPiton checkout (idempotent – safe to re-run):
bash scripts/patch_openpiton.sh /mnt/c/Users/Potato/Desktop/openpiton
```

6.3 Running the test suite

Tier 0 (no Ray, no hardware, no LLM) — the everyday sanity check:

```
pytest chia_openpiton/test mace/test -q \
  --ignore=chia_openpiton/test/cluster --ignore=mace/test/cluster
```

Verified directly while preparing this document: **309 passed, 0 failed, ~9 seconds**.

Tier 1 (real Ray, stubbed hardware) — lives under each package's `test/cluster/` directory; needs a local Ray instance advertising the right custom resources (`openpiton`, `fake_creds`, etc. — see each test module's own

docstring for the exact resource shape it expects).

Tier 2 (real hardware) and the GCP acceptance test — gated by environment variables so they never run by accident:

Variable	Gates
<code>OPENPITON_TEST_REAL=1</code>	Real Ariane/SPARC builds against a real <code>PITON_ROOT</code> (needs Verilator + the RISC-V toolchain already built)
<code>OPENPITON_TEST_GCP=1</code>	GCP acceptance test #2 — must run from the cluster's head node with <code>chia up cluster/local.yaml</code> already active
<code>OPENPITON_TEST_CORE</code>	Overrides which core tier-2 tests target (default <code>ariane</code>)
<code>MACE_LLM</code>	<code>opencode</code> (default) / <code>claude</code> / <code>antigravity</code> / <code>vertex</code> — which real LLM backend <code>mace.llm.make_llm()</code> builds

6.4 Running a real end-to-end demo

`examples/mace_end_to_end.py` is the real, runnable, plan → execute → metrics driver:

```
conda activate chia_env
python examples/mace_end_to_end.py \
  --piton-root /mnt/c/Users/Potato/Desktop/openpiton \
  --model opencode/big-pickle \
  --max-iterations 3
```

Add `--piton-root-2 <second checkout>` to exercise real parallel fan-out across two checkouts. Needs a real OpenPiton checkout with the toolchain already built (Section 6.2's patch script applied) and a working `opencode` backend. Prints per-iteration, per-task results, any triage diagnoses, and the five summary metrics at the end; everything is also recorded to the SQLite DB at `--db-path` (default `runs/mace_end_to_end.db`).

Two smaller, LLM-light demos also exist: `examples/hello_openpiton.py` (an agent edits one line of testbench RTL, then a build+run confirms the marker via the simulator's own transcript — the smallest possible “agent touches real hardware” demo, predates the full loop) and `examples/run_barrier_atomic.py` (no LLM at all — a direct configure→build→run of the `barrier_atomic.c` gate workload; see Section 7 on its status).

6.5 The cluster

```
export HEAD_IP=$(hostname -I | awk '{print $1}')
export GCP_PROJECT=mace-508004
export GCP_SSH_KEY=$HOME/.ssh/id_ed25519
export TS_AUTHKEY=<tailscale reusable auth key>

chia up cluster/local.yaml
# ... work ...
chia down cluster/local.yaml
```

Requires `gcloud auth application-default login` plus `gcloud auth application-default set-quota-project $GCP_PROJECT` once, and the Compute Engine API enabled on the project. See `cluster/local.yaml`'s own header comments for the full, current list of prerequisites and the real quota/stockout gotchas hit getting this far (Section 4).

7. Known rough edges and standing conventions

These are conventions the project has followed consistently, not accidents — worth knowing so you don't second-guess them or accidentally break them.

- **No AI co-author trailers in commits.** A deliberate, repeated instruction from the project owner. Every commit in the history is authored normally with no `Co-Authored-By` trailer for AI assistance, despite this being AI-assisted work throughout — keep following this convention for any new commits.
- **Small, focused commits directly on `main`, no branches or PRs.** The git log is a long, linear chain of small, individually-tested commits, each with a detailed, honest commit message describing what changed and *why* (they are worth reading directly — `git log --stat` — for context this document doesn't repeat). As of this writing the local `main` is 8 commits ahead of `origin/main`, not yet pushed.
- **An unexplained, untracked file:** `examples/run_barrier_atomic.py`. It is well-formed and functional-looking (a direct, LLM-free `configure→build(clean=True)→run(barrier_atomic.c)` driver), but nobody has claimed authorship and it has never been added to git. It has been deliberately left alone rather than deleted or committed on someone else's behalf — do the same unless you can confirm with the team who it belongs to and whether it should be kept.
- **The OpenPiton checkout itself carries a lot of its own uncommitted state** — CRLF/tooling-file normalization noise, the pyHP preprocessor's `.tmp.v` build byproducts written back into the source tree on every build, and (as of this writing) the symlink-related changes from Section 5's in-flight patch. This is expected and long-standing, not a new problem — but it does mean `git status` inside `Desktop/openpiton` will look alarming at first glance; read it against this context rather than assuming something is broken.
- **Build/output artifacts are gitignored, not missing:** `mace.egg-info/`, `runs/` (loop run databases and logs), `.pytest_cache/`, `*.db`. `.env` (site-specific values — GCP project, paths, auth keys) is also gitignored deliberately and never committed; ask the project owner for its contents rather than trying to reconstruct it.
- **Ownership.** The original plan document assigns a specific split of responsibilities between two people (adapter/hardware-facing work vs. cluster/LLM/loop-driver work), but the project has moved a long way since that was written and the visible git history shows one author throughout — confirm the current division of labor directly with the project owner rather than assuming the original split still holds.

End of document. This was assembled by reading the project's living planning document in full, the actual current source of every module described in Section 2, the full commit history, the current (including uncommitted) working-tree state of both the MACE and OpenPiton checkouts, and by directly running the tier-0 test suite — not summarized from an earlier summary.